EEN1095 Literature and Reference Log

Student Name	Darshan Mahadeva Naika	**Student ID**	A00090581
Project Title	Design and Evaluation of Low-Sensitivity Digital Filters Implemented in Python Under Varying Numerical Precision Levels	**Supervisor**	Martin Collier
Implementation Period	Week 1 - Week 2	**Version Date**	22/05/2026

Guidance: Maintain this as one cumulative document throughout EEN1095. Update it every two weeks and do not remove earlier records. Record progress in a timely and concise manner. Where no change has occurred, state this explicitly.

Part A. Log of literature searches performed

Period	Date	Database	Search Terms	Results Found	Useful Hits
Week 1-2	15/05/2026	Google Scholar / Nature	SciPy NumPy Python scientific computing floating point arithmetic precision control	38	2
Week 1-2	18/05/2026	IEEE Xplore	Robust IIR filter synthesis finite precision floating point Python implementation	22	2

Part B. Log of key articles reviewed

Article Review 1

Reporting period	Week 1-2 (May 2026)
Article citation (author, title, source, volume, pages, date, DOI)	P. Virtanen, R. Gommers, T. E. Oliphant et al., SciPy 1.0: Fundamental algorithms for scientific computing in Python, Nature Methods, vol. 17, pp. 261-272, Feb. 2020, doi: 10.1038/s41592-019-0686-2.
Key findings	Describes SciPy as the de facto standard for scientific computing in Python. scipy.signal provides IIR and FIR filter design functions including Butterworth, Chebyshev, and elliptic designs. SOS format explicitly recommended for numerical accuracy in high-order IIR filter implementations.
Strengths and weaknesses	Strengths: Peer-reviewed published reference for my primary filter design tool. SOS format recommendation directly addresses coefficient sensitivity - relevant to my research goals. Weaknesses: Does not address multi-precision arithmetic or structural comparison of filter implementations.
Relevance to my project	SciPy is the filter design tool set up and validated in Week 1-2. I used scipy.signal to run pilot filter designs for Butterworth and elliptic IIR filters during framework setup and testing.
How this paper informed my design / implementation / evaluation	All filter designs in my framework use SciPy scipy.signal. The SOS output format is used to minimise numerical error during the design stage before structural and precision comparison experiments begin. Set up and validated in Week 1-2.

Article Review 2

Reporting period	Week 1-2 (May 2026)
Article citation (author, title, source, volume, pages, date, DOI)	C. R. Harris, K. J. Millman, S. J. van der Walt et al., Array programming with NumPy, Nature, vol. 585, pp. 357-362, Sept. 2020, doi: 10.1038/s41586-020-2649-2.
Key findings	Foundational reference for NumPy array-based floating-point arithmetic. NumPy natively supports float16, float32, and float64 as selectable data types via the dtype parameter, enabling direct and explicit control of arithmetic precision in scientific computing.
Strengths and weaknesses	Strengths: Peer-reviewed published reference for my primary arithmetic engine. Native multi-precision dtype support is essential to my framework - without NumPy dtype selection my precision control mechanism does not exist. Weaknesses: Does not address digital filter design or structural comparison.
Relevance to my project	NumPy provides float16, float32, and float64 - three of my four precision levels. The precision control for these three levels was set up and validated in Week 1-2 using NumPy dtype selection.
How this paper informed my design / implementation / evaluation	NumPy dtype selection is the mechanism by which I control arithmetic precision in my framework. Converting SciPy-designed float64 coefficients to float16 or float32 via NumPy astype() is central to my experimental method. Validated in Week 1-2.

Article Review 3

Reporting period	Week 1-2 (May 2026)
Article citation (author, title, source, volume, pages, date, DOI)	H. Ko and J. Tsai, Robust and computationally efficient digital IIR filter synthesis and stability analysis under finite precision implementations, IEEE Transactions on Signal Processing, vol. 68, pp. 1807-1822, Mar. 2020, doi: 10.1109/TSP.2020.2977848.
Key findings	Studied IIR filter robustness under finite floating-point precision and showed that structure and precision level jointly determine stability. Proposed a robust IIR synthesis method explicitly accounting for finite-precision stability under floating-point implementations.
Strengths and weaknesses	Strengths: Most directly related recent paper to my research question. Addresses software floating-point precision in Python-based environments, peer-reviewed IEEE publication. Weaknesses: Studied in a fixed structural context without cross-structure systematic comparison.
Relevance to my project	Provides the theoretical motivation for the precision control framework built in Week 1-2. Ko and Tsai confirm that testing IIR filters across different floating-point precision levels is a meaningful and important research activity.
How this paper informed my design / implementation / evaluation	Informed the design of the precision control strategy in my framework - specifically why float16, float32, float64, and mpmath were chosen as the four test levels. Defines the specific research gap my project addresses by adding cross-structure comparison.

Article Review 4

Reporting period	Week 1-2 (May 2026)
Article citation (author, title, source, volume, pages, date, DOI)	A. V. Oppenheim and R. W. Schafer, Discrete-Time Signal Processing, 3rd ed. Upper Saddle River, NJ, USA: Prentice-Hall, 2010, ISBN: 978-0-13-198842-2.
Key findings	Standard DSP textbook covering sampling theory, quantisation, finite word-length effects, pole-zero analysis, frequency response computation, and IIR and FIR filter structures comprehensively. Establishes all foundational DSP concepts upon which this research builds.
Strengths and weaknesses	Strengths: Comprehensive foundational DSP reference covering all theoretical concepts needed for framework design - sampling, quantisation, filter design, pole-zero analysis. Widely recognised standard textbook. Weaknesses: Does not cover recent Python-based or software floating-point specific research.
Relevance to my project	Provided the foundational DSP theory needed to design and implement the Python framework in Week 1-2. The definitions of pole displacement, stability margin, frequency response deviation, and roundoff noise used in the framework all come from this textbook.
How this paper informed my design / implementation / evaluation	Used as the theoretical reference for all framework design decisions in Week 1-2. The frequency response computation across 4096 points, pole-zero calculation from denominator polynomial, and the five performance metric definitions are all grounded in Oppenheim and Schafer.